SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	**5**
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	**5**
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

EMPLOYEE Table

EmpID	EmpName	Salary	DeptID
101	Arjun	55000	D1
102	Meera	48000	D2
103	Karan	72000	D1
104	Divya	60000	D3
105	Ravi	45000	D2

DEPARTMENT Table

DeptID	DeptName	Location
D1	CSE	Block A
D2	ECE	Block B
D3	MECH	Block C

STUDENT Table

SID	Name	DOB	DeptID
S01	Hari	2004-05-10	D1
S02	Priya	2003-11-22	D2
S03	Manoj	2004-01-15	D1
S04	Kavya	2003-07-09	D3

COURSE Table

CourseID	CourseName
C1	DBMS
C2	AI
C3	Networks

ENROLLMENT Table

SID	CourseID
S01	C1
S01	C2
S01	C3
S02	C1
S03	C1
S03	C2
S04	C3

PROJECT Table

ProjID	ProjName	EmpID
P1	Billing System	101
P2	AI Chatbot	103
P3	Web Portal	102
P4	IoT Device	104

SALES Table

SaleID	Product	Quantity	Price
1	Pen	10	15
2	Book	5	120
3	Pen	20	15
4	Bag	2	500
5	Book	3	120

Question 1:

1. Selection (σ)
Example: Employees with salary greater than 50000

$$\sigma_{Salary>50000}(Employee)$$

2. Projection (π)
Example: List only employee names and salaries

$$\pi_{EmpName,Salary}(Employee)$$

3. Cartesian product (×)
Example: All possible Employee–Department pairs

$$Employee \times Department$$

4. Join (⋈)
Example: Employees with their department names

$$Employee \bowtie_{Employee.DeptID=Department.DeptID} Department$$

5. Union (∪)
Example: Employees in departments D1 or D2

$$\sigma_{DeptID='D1'}(Employee) \cup \sigma_{DeptID='D2'}(Employee)$$

6. Difference (−)
Example: Departments that have no employees
Let:

$$UsedDepts = \pi_{DeptID}(Employee)$$
$$AllDepts = \pi_{DeptID}(Department)$$

Then:

$$AllDepts - UsedDepts$$
$$Department - (Department \bowtie Employee)$$

These expressions demonstrate applying **σ, π, ∪, −, ×, ⋈** on Employee and Department as required.

Question 2:

1. Create DEPARTMENT Table

```
CREATE TABLE DEPARTMENT (
   DeptID     VARCHAR(10) PRIMARY KEY,
   DeptName   VARCHAR(50) NOT NULL UNIQUE
);
```

Constraints used:

- PRIMARY KEY → DeptID
- NOT NULL → DeptName
- UNIQUE → No two departments can have the same name

2. Create STUDENT table

```
CREATE TABLE STUDENT (
   SID      VARCHAR(10) PRIMARY KEY,
   Name     VARCHAR(50) NOT NULL,
   DOB      DATE NOT NULL,
   DeptID   VARCHAR(10),
   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

Constraints used:

- PRIMARY KEY → SID
- NOT NULL → Name, DOB
- FOREIGN KEY → DeptID references DEPARTMENT(DeptID)

Question 3:

1. COUNT — Number of sales per product

```
SELECT Product, COUNT(*) AS TotalSales
FROM Sales
GROUP BY Product;
```

2. SUM — Total quantity sold per product

```
SELECT Product, SUM(Quantity) AS TotalQuantity
FROM Sales
GROUP BY Product;
```

3. AVG — Average quantity sold per product

```
SELECT Product, AVG(Quantity) AS AvgQuantity
FROM Sales
```

```
GROUP BY Product;
```

4. MAX — Highest quantity sold for any product

```
SELECT Product, MAX(Quantity) AS MaxQuantity
FROM Sales
GROUP BY Product;
```

5. MIN — Lowest quantity sold for any product

```
SELECT Product, MIN(Quantity) AS MinQuantity
FROM Sales
GROUP BY Product;
```

6. SUM with HAVING — Products with total quantity > 20

```
SELECT Product, SUM(Quantity) AS TotalQuantity
FROM Sales
GROUP BY Product
HAVING SUM(Quantity) > 20;
```

7. Revenue calculation (Quantity × Price) per product

```
SELECT Product, SUM(Quantity * Price) AS TotalRevenue
FROM Sales
GROUP BY Product;
```

8. HAVING with AVG — Products whose average quantity > 10

```
SELECT Product, AVG(Quantity) AS AvgQty
FROM Sales
GROUP BY Product
HAVING AVG(Quantity) > 10;
```

9. COUNT + HAVING — Products sold more than 2 times

```
SELECT Product, COUNT(*) AS SaleCount
FROM Sales
GROUP BY Product
HAVING COUNT(*) > 2;
```

10.
```
SELECT Product,
COUNT(*) AS TotalSales,
SUM(Quantity) AS TotalQty,
```

```
AVG(Quantity) AS AvgQty,
MAX(Quantity) AS MaxQty,
MIN(Quantity) AS MinQty
FROM Sales
GROUP BY Product;
```

Question 4:

Correlated Subquery: Employees earning more than the average salary of their own department

```
SELECT EmpID, EmpName, Salary, DeptID
FROM Employee E
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employee
    WHERE DeptID = E.DeptID
);
```

Why this is a correlated subquery?

Because the inner query:

```
SELECT AVG(Salary)
FROM Employee
WHERE DeptID = E.DeptID
```

depends on the current row of the outer query (E.DeptID). It calculates the average salary of that employee's department, and then the outer query checks if the employee's salary is greater than that average.

What this query returns

Employees whose salary is higher than the average salary of their own department.

Question 5:

1. INNER JOIN

```
SELECT E.EmpID, E.EmpName, P.ProjID, P.ProjName
FROM Employee E
INNER JOIN Project P
```

```
ON E.EmpID = P.EmpID;
```

- Returns only matching rows
- Employees without projects are excluded

2. LEFT OUTER JOIN

   All employees, even those with no project

```
SELECT E.EmpID, E.EmpName, P.ProjID, P.ProjName
FROM Employee E
LEFT JOIN Project P
ON E.EmpID = P.EmpID;
```

- Shows all employees
- Employees without projects show NULL for project fields

3. RIGHT OUTER JOIN

   all projects, even those not assigned to any employee

```
SELECT E.EmpID, E.EmpName, P.ProjID, P.ProjName
FROM Employee E
RIGHT JOIN Project P
ON E.EmpID = P.EmpID;
```

- Shows all projects
- Projects without employees show NULL for employee fields

4. FULL OUTER JOIN

   All employees + all projects (MySQL doesn’t support FULL JOIN directly) (Use UNION of LEFT + RIGHT)

```
SELECT E.EmpID, E.EmpName, P.ProjID, P.ProjName
FROM Employee E
LEFT JOIN Project P
ON E.EmpID = P.EmpID

UNION

SELECT E.EmpID, E.EmpName, P.ProjID, P.ProjName
FROM Employee E
```

```
RIGHT JOIN Project P
ON E.EmpID = P.EmpID;
```

- Shows everything
- Employees without projects
- Projects without employees

5. CROSS JOIN

   All possible combinations of Employee × Project

```
SELECT E.EmpID, E.EmpName, P.ProjID, P.ProjName
FROM Employee E
CROSS JOIN Project P;
```

- Cartesian product
- Every employee paired with every project

Question 6:

Find names of students who are enrolled in all courses offered

- STUDENT(SID, Name, DOB, DeptID)
- COURSE(CourseID, CourseName)
- ENROLLMENT(SID, CourseID)

Concept:

A student is enrolled in *all* courses if:

The set of courses they have taken = the set of all courses offered.

This is a division problem, but since your question restricts you to σ, π, ×, ⋈, ∪, −, we express it using set difference.

Step-by-step Relational Algebra

1. **All courses offered**

$$AllCourses = \pi_{CourseID}(Course)$$

2. **.Courses taken by each student**

$$\text{StudentCourses}=\pi\text{SID,CourseID(Enrollment)}$$

3. **Students missing at least one course**

$$\text{Missing}=\pi\text{SID(AllCourses}\times\pi\text{SID(Student)} - \text{StudentCourses)}$$

4. **Students who are NOT missing any course**

$$AllStudents = \pi_{SID}(Student)$$
$$Complete = AllStudents - Missing$$

5. **Final answer: Names of students enrolled in all courses**

$$\pi_{Name}(Complete \bowtie Student)$$

πName((πSID(Student)−πSID((πCourseID(Course)×πSID(Student))−πSID,CourseID(Enrollment)))⋈Student)

Question 7:

Find names of students who are enrolled in *all* courses offered

We use the standard relations:

- STUDENT(SID, Name, DOB, DeptID)
- COURSE(CourseID, CourseName)
- ENROLLMENT(SID, CourseID)

Your assessment expects the division logic, but since division is not allowed, we express it using set difference.

Final Relational Algebra Expression

$$\pi_{Name}((\pi_{SID}(Student) - \pi_{SID}((\pi_{CourseID}(Course) \times \pi_{SID}(Student)) - \pi_{SID,CourseID}(Enrollment))) \bowtie Student)$$

Simple Explanation (for your understanding)

1. **AllCourses** = all CourseIDs

$$\pi_{CourseID}(Course)$$

2. **StudentCourses** = all (SID, CourseID) pairs

$$\pi_{SID,CourseID}(Enrollment)$$

3. **Missing** = students who are missing at least one course

$$\left(\pi_{CourseID}(Course) \times \pi_{SID}(Student)\right) - \pi_{SID,CourseID}(Enrollment)$$

4. **CompleteStudents** = students who are *not* missing any course

$$\pi_{SID}(Student) - Missing$$

5. **Final names**

$$\pi_{Name}(CompleteStudents \bowtie Student)$$

Question 8:

SQL DML Statements (INSERT, UPDATE, DELETE) with Integrity Constraints

DEPARTMENT(DeptID PRIMARY KEY, DeptName UNIQUE) STUDENT(SID PRIMARY KEY, Name NOT NULL, DOB NOT NULL, DeptID FOREIGN KEY)

INSERT

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
VALUES ('S01', 'Arun', '2004-05-10', 'D1');
```

UPDATE

```
UPDATE STUDENT
SET DeptID = 'D1'
WHERE SID = 'S01';
```

DELETE

```
DELETE FROM STUDENT
WHERE SID = 'S01';
```

Question 9:

Tuple Relational Calculus (TRC)

Find all employees who work in the 'IT' department and earn more than 50000
Relations:

- Employee(EmpID, EmpName, Salary, DeptID)
- Department(DeptID, DeptName)

TRC Expression

{e.EmpName |
Employee(e)∧e.Salary>50000∧∃d(Department(d)∧d.DeptName='IT'∧d.DeptID=e.DeptID)}

- Uses existential quantifier
- Matches IT department
- Salary > 50000
- Returns employee names

Question 10:

Airline Reservation System — Relational Schema + 5 SQL Queries

Relational Schema

AIRPORT
- AirportID (PK)
- AirportName
- City
- Country

FLIGHT
- FlightID (PK)
- SourceAirport (FK → AIRPORT)
- DestinationAirport (FK → AIRPORT)
- DepartureTime
- ArrivalTime

PASSENGER
- PassengerID (PK)
- Name
- Age

RESERVATION
- ReservationID (PK)
- PassengerID (FK → PASSENGER)
- FlightID (FK → FLIGHT)
- SeatNo
- BookingDate

Five SQL Queries (Joins, Subqueries, Views)

1. INNER JOIN — Passenger + Flight

```
SELECT P.Name, F.FlightID, F.DepartureTime
FROM PASSENGER P
INNER JOIN RESERVATION R ON P.PassengerID = R.PassengerID
INNER JOIN FLIGHT F ON R.FlightID = F.FlightID;
```

2.. LEFT JOIN — All flights even without reservations

```
SELECT F.FlightID, R.ReservationID
FROM FLIGHT F
LEFT JOIN RESERVATION R ON F.FlightID = R.FlightID;
```

3.. SUBQUERY — Passengers who booked more than 2 flights

```
SELECT Name
FROM PASSENGER
WHERE PassengerID IN (
        SELECT PassengerID
        FROM RESERVATION
        GROUP BY PassengerID
        HAVING COUNT(*) > 2
);
```

4.. SUBQUERY — Flights departing from airports in India

```
SELECT FlightID
FROM FLIGHT
WHERE SourceAirport IN (
         SELECT AirportID
        FROM AIRPORT
        WHERE Country = 'India'
);
```

5.. VIEW — Flight Reservation Summary

```
CREATE VIEW FlightSummary AS
SELECT F.FlightID,
         COUNT(R.ReservationID) AS TotalBookings
FROM FLIGHT F
LEFT JOIN RESERVATION R ON F.FlightID = R.FlightID
GROUP BY F.FlightID;
```

6.. Query on the View

```
SELECT * FROM FlightSummary
WHERE TotalBookings > 50;
```
